

palmsource™

PalmSource Seminar Series
Build Tomorrow ▸▸▸ Today

**Presented By:**
**Owen Emry,**
**DTS Engineer**

palm™
POWERED

# Palm OS 5 and ARM

# Agenda

1. Introduction to PACE
2. Getting compatible in the PACE environment
3. One step further:  taking advantage of ARM
4. New tools for ARM development

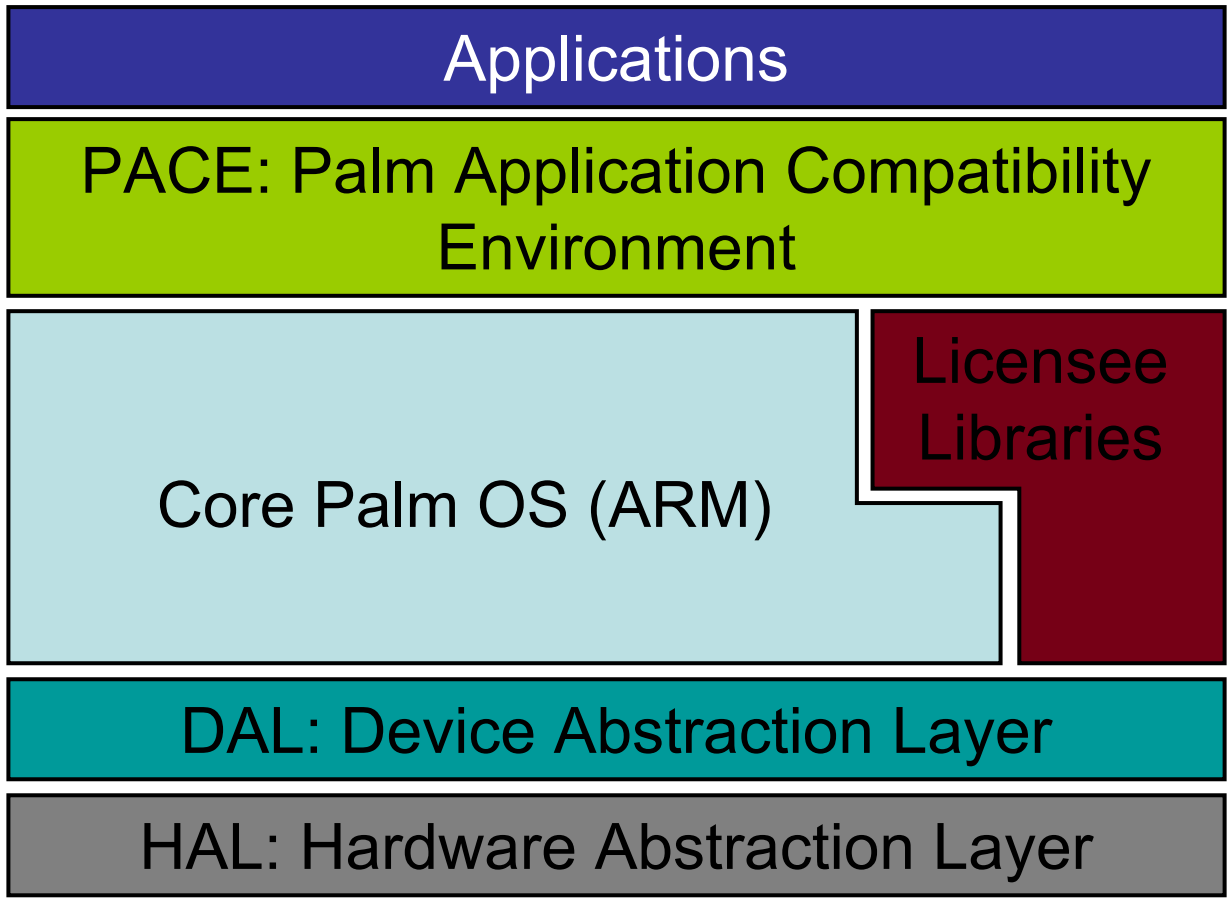
# Introducing PACE
## *Palm Application Compatibility Environment*

- **Performs 68K emulation, but it's much more than just an emulator**

**PACE also:**

- **Handles native OS/68K interaction**
  - Byteswapping (endianness) issues
  - Structure translation
  - Resource translation
  - Shadow constructs
  - Parameter conversion
- **Implements 68K debugger nub**

# Palm OS 5 Block Diagram



# Better compatibility benefits the Palm OS economy

**For the software vendor:**

- **Less work tomorrow**
- **Run on more devices**
  - More potential customers = more sales
- **Run on future devices**
  - Happier customers (when they upgrade devices)
  - Happy customers = loyal customers

# UI "shadow" structures

- **Created by PACE for compatibility**
- **Palm OS uses ARM UI structures internally**
- **PACE creates 68K "shadow" structures for 68K apps to use**
- **But PACE might not fill in all the shadow's members, so treat the shadows as opaque (and use Palm OS Glue)**
- **Debug ROMs fill the shadow with 0xFE**
  - This can make some bit fields look plausible

# Technical details about PACE

- **Two implementations: Fast and Slow PACE**
- **“Nice” and “Mean” PACE**
- **Separate threads (e.g. sound) have separate PACE states**
- **Most OS calls are thread-safe**

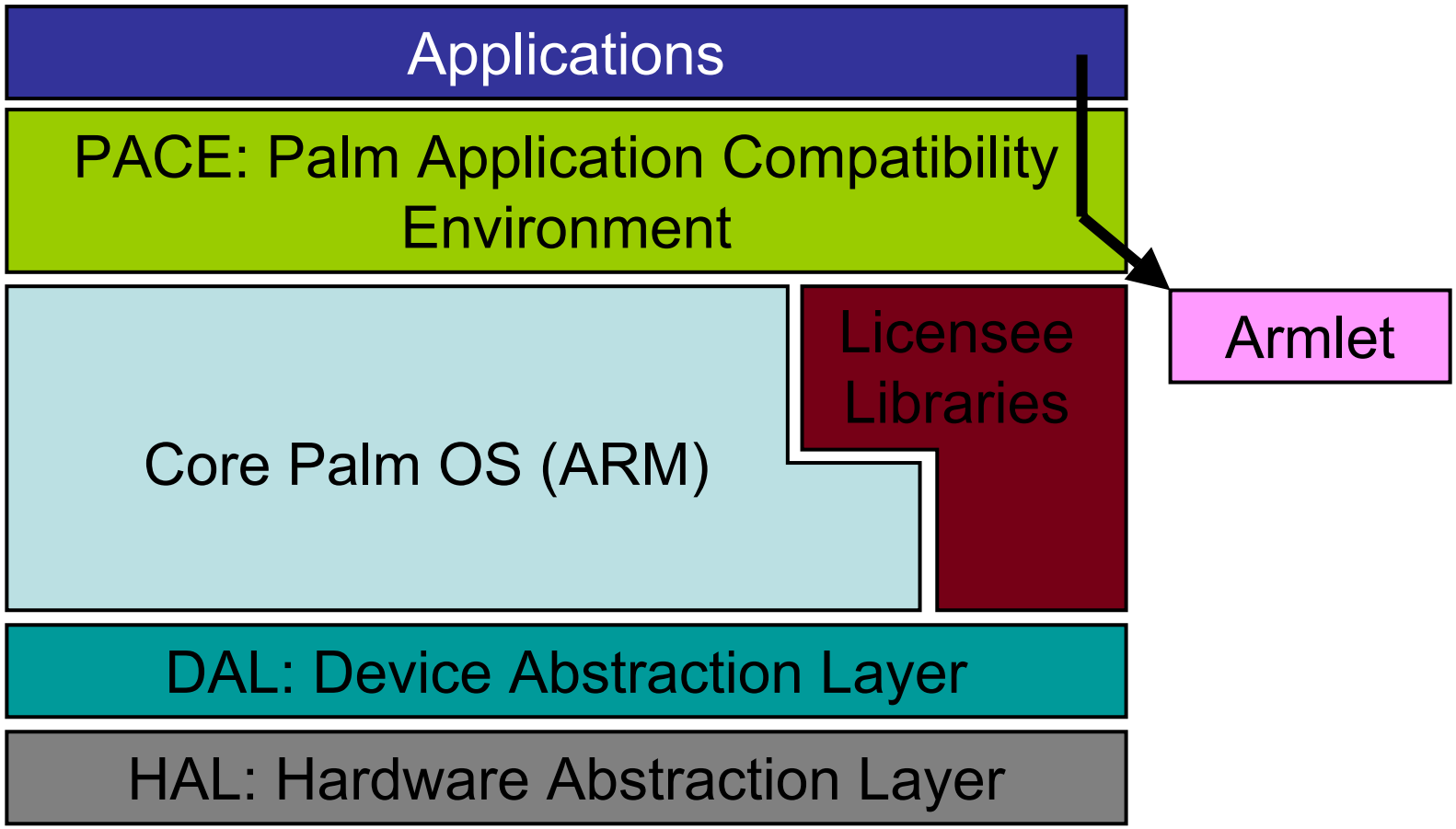
# Advanced pitfalls

- **PACE isn't around when an ARM app (e.g. HotSync) is running, so background apps can fail while doing UI**
  - So do background UI after sub launching yourself
- **Devices with more memory can expose string-related errors**

# Introducing armlets

- **Armlets are a mechanism to enable performance-critical applications to make subroutine calls to ARM code – code that isn't emulated**

# Palm OS 5 Block Diagram



# PceNativeCall: PACE's door to ARM code

```
UInt32 PceNativeCall(
  NativeFuncType *nativeFuncP,
  void *userDataP)
```

**nativeFuncP is a pointer to the armlet's entry point**

**userDataP is a pointer to some data to share with the armlet**

# ARM function prototype

```
#include "PceNativeCall.h"

unsigned long NativeFunction(
  const void       *emulStateP,
        void       *userData68KP,
  Call68KFuncType *call68KFuncP)

{
  return (unsigned long)userData68KP;
}
```

# Armlet steps

1. Compile + link the armlet code
2. Extract the code section to a .bin file
3. Include the .bin as a resource in your app
4. Lock down the armlet resource and invoke with PceNativeCall

# Pitfalls

- **Endian issues**
- **Structure packing**
- **Natural alignment**
- **Context (globals, etc.) not provided by system**

# Demos: New ARM tools

# Questions?